Unité 2 — Sécurité mémoire & débordement de pile

Livrable B — Support de cours (version enrichie) · UE « Software Security »

Enseignement supérieur de promotion sociale · Code UE 7532 14 U32 D1 · Document enseignant/étudiant · v2 — 11 octobre 2026

> **Place dans le programme.** Cette unité couvre le point du programme « expliquer et mettre en pratique, sur des systèmes vulnérables, les failles les plus courantes en sécurité logicielle, et comprendre comment ces problèmes peuvent être évités ». Elle pose les fondations de l'exploitation de binaires : organisation de la mémoire, pile et convention d'appel, débordement de tampon, passage du plantage à l'exécution de code, puis protections mémoire et remédiations. Les laboratoires associés (cahier étudiant séparé) reprennent la progression `stack0`→`stack3` du support de référence *(meltypot, études de cas stack0–stack3 et « Getting a shell »)*.

1. Pourquoi la mémoire est au cœur de la sécurité logicielle

Les langages C et C ++ ne sont pas *sûrs en mémoire* par défaut : le programmeur peut lire ou écrire en dehors des objets alloués. Microsoft et le projet Chromium rapportent tous deux qu'environ **70 % des failles de gravité élevée** dans leurs bases de code C/C++ sont des problèmes de sûreté mémoire *(meltypot, « A Short History of Famous Incidents »)*. Comprendre ces bugs, savoir les observer expérimentalement, puis savoir les corriger, est donc une compétence centrale.

Un programme est **sûr en mémoire** si chaque accès ne touche que des objets : **spatialement valides** (dans les bornes de l'objet alloué : pas de hors-bornes) et **temporellement valides** (encore vivants au moment de l'accès : pas d'`use-after-free`, pas de lecture avant initialisation). Cette unité traite la violation spatiale la plus classique : le **débordement de tampon sur la pile**.

2. Le modèle mémoire d'un processus

À l'exécution, un processus C voit son espace d'adressage découpé en segments. Des adresses basses vers les hautes :

Segment	Contenu	Remarques sécurité
`.text`	Code machine compilé	En lecture seule / exécutable. Cible des détournements de flot.
`.data` / `.bss`	Variables globales / statiques	Inscriptible, non exécutable par défaut.
`heap` (tas)	`malloc` / `new`, croît vers le haut	Débordements de tas, use-after-free (Unité 3).
`stack` (pile)	Variables locales, adresses de retour ; croît vers le **bas**	**Cœur de cette unité.** Contient l'adresse de retour que l'attaquant veut écraser.

Point essentiel : la pile croît vers les adresses **basses**, mais `strcpy` et consorts écrivent vers les adresses **hautes**. Un tampon trop petit déborde donc *vers* les données sauvegardées juste après lui (pointeur de trame et adresse de retour).

3. La pile et la convention d'appel x86 (32 bits)

3.1 Registres utiles

Registre	Rôle
`eax`	Accumulateur (résultats, numéro d'appel système).
`ebx,ecx,edx`	Usage général ; arguments d'appels système.
`ebp`	**Base Pointer** : base de la trame courante.
`esp`	**Stack Pointer** : sommet de la pile.
`eip`	**Instruction Pointer** : adresse de la prochaine instruction. **C'est lui que l'attaquant veut contrôler.**

Tables des registres et des instructions adaptées de meltypot (« x86 Registers », « Common x86 Assembly Instructions »).

3.2 Instructions clés

`push` (empile), `pop` (dépile), `call addr` (empile `eip` puis saute), `ret` (dépile dans `eip`), `leave` (`mov esp,ebp ; pop ebp`), `mov/lea`, `int 0x80` (appel système), `nop` (ne fait rien).

3.3 Anatomie d'une trame : ce qui se passe à l'appel d'une fonction

1. Les paramètres sont empilés (en x86 32 bits).
2. `call` empile l'adresse de retour (l'`eip` suivant) puis saute dans la fonction.
3. Le prologue empile l'ancien `ebp` (`push ebp ; mov ebp,esp`).
4. De la place est réservée pour les variables locales (`sub esp, N`).

La trame ressemble alors à ceci (adresses hautes en haut) :

SCHÉMA — TRAME DE PILE D'UNE FONCTION X86 32 BITS

```
adresses hautes
+----------------------+
|  parametres (a,b,c)  |   <- [ebp+8], [ebp+12], [ebp+16]
+----------------------+
|  adresse de retour   |   <- [ebp+4]   (l'EIP sauve par 'call')
+----------------------+
|  ancien EBP sauve    |   <- [ebp]
+----------------------+
|  variables locales   |   <- [ebp-4], ... (le tampon vit ici)
+----------------------+
adresses basses  (esp)
```

3.4 Observation réelle (programme `stack0`)

Le labo 2.1 compile un petit programme `sum(1,2,3)` et observe sa trame sous GDB. Ce que les étudiants obtiennent réellement :

SORTIE GDB RÉELLE (ENVIRONNEMENT DE RÉFÉRENCE)

```
Breakpoint 1, sum (a=1, b=2, c=3) at stack0.c:4
eip=0x804916c  esp=0xffffda9c  ebp=0xffffdaa0
---- pile depuis EBP : [ebp]=ebp sauve  [ebp+4]=retour  [ebp+8/12/16]=a,b,c ----
0xffffdaa0:  0xffffdab8  0x08049192  0x00000001  0x00000002
0xffffdab0:  0x00000003  0x00000000
```

On lit directement la théorie dans la mémoire : à `[ebp]` l'ancien `ebp` (`0xffffdab8`), à `[ebp+4]` l'adresse de retour `0x08049192` (exactement l'instruction qui suit le `call sum` dans `main`), puis les trois paramètres `1, 2, 3`. **L'adresse de retour est une donnée en mémoire : si on peut écrire par-dessus, on détourne le programme.**

4. Les fonctions C dangereuses

Certaines fonctions de la bibliothèque standard copient sans vérifier la taille de destination : *(meltypot, « Unsafe C functions »)*

Fonction	Problème

`gets(buf)`	Lit une ligne sans aucune limite. À proscrire absolument.
`strcpy(dst,src)`	Copie jusqu'au `\0` sans borne sur `dst`.
`strcat(dst,src)`	Concatène sans borne.
`sprintf(buf,fmt,...)`	Écrit un format sans borne.

Alternatives bornées : `fgets`, `strncpy`/`strlcpy`, `snprintf` — en vérifiant toujours la taille de destination. Ce point est approfondi en Unité 9 (développement sécurisé).

5. Le débordement de tampon sur la pile

L'exemple canonique *(meltypot, « Stack Buffer Overflow »)* :

C — DÉBORDEMENT CLASSIQUE

```
void greet(char *name) {
    char buf[16];
    strcpy(buf, name);      // aucun controle de longueur !
    printf("Hello, %s\n", buf);
}
int main(int argc, char **argv) {
    greet(argv[1]);         // l'attaquant controle argv[1]
}
```

Passez un `name` de plus de 16 octets et `strcpy` écrit joyeusement au-delà de `buf`. Juste après `buf` sur la pile se trouvent des cibles intéressantes : le **pointeur de trame sauvé** puis l'**adresse de retour** de `greet`.

Trois régimes selon la longueur de l'entrée, que les étudiants observent concrètement avec `stack1` (variable `value` juste après un tampon de 16 octets) :

SORTIE RÉELLE — STACK1 (OFFSET TAMPON→VALUE = 16)

```
len=16  : value est inchangee (0x00000000)      <- on remplit juste le tampon
len=17  : value a ete modifiee -> 0x00000041      <- 1 octet deborde ('A')
len=20  : value a ete modifiee -> 0x41414141      <- 4 octets : value entierement ecrasee
```

Même principe, un cran plus loin : en continuant de déborder, on atteint l'**ancien EBP** puis l'**adresse de retour**. Avec `stack3` (tampon de 32 octets), l'adresse de retour est atteinte après exactement **36 octets** (32 de tampon + 4 d'EBP sauvé) :

SORTIE RÉELLE — MESURE DE L'OFFSET EIP (A*N + 'BBBB')

```
offset 32 -> (pas de controle d'EIP)
offset 36 -> EIP=0x42424242        <- 'BBBB' atterrit pile sur l'adresse de retour
offset 40 -> EIP=0x41414141
```

Point méthodologique. L'offset exact (ici 36) dépend du compilateur et des options. Les étudiants ne l'apprennent pas par cœur : ils le *mesurent* (motif cyclique, observation du registre au plantage). C'est la compétence visée.

6. Du plantage à l'exécution de code

Un débordement naïf corrompt l'adresse de retour et le programme plante quand `ret` saute vers une adresse invalide. L'attaquant fait mieux *(meltypot, « From Crash to Code Execution »)* :

1. Placer du **shellcode** (instructions machine brutes) dans le tampon, précédé d'un **NOP sled**.
2. Écraser l'adresse de retour par (une approximation de) l'adresse du tampon.
3. Au `ret`, le CPU charge cette adresse, glisse sur les `NOP`, puis exécute le shellcode.

SCHÉMA — NOP SLED + SHELLCODE

```
 NOP NOP NOP NOP NOP | shell shell shell | EBP sauve | adr. de retour (ecrasee -> &buf)
 \_____ NOP sled ____/ \___ shellcode ___/
           au 'ret', le CPU saute dans le sled puis tombe sur le shellcode
```

6.1 Qu'est-ce qu'un shellcode ?

Du code machine, souvent destiné à lancer un interpréteur de commandes (un *shell*). Sous Linux i386, obtenir un shell revient à invoquer l'appel système `execve("/bin/sh", ...)` *(meltypot, « Getting a shell »)*. En assembleur :

1. Numéro d'appel système dans `eax` (`execve` = 11 = `0x0b`).
2. 1er paramètre dans `ebx`, 2e dans `ecx`, 3e dans `edx`.
3. Déclencher l'interruption `int 0x80`.

Contraintes : pas d'octet nul `\x00` (il stoppe `strcpy`), et l'on évite espaces (`0x20`) et sauts de ligne (`0x0a`) selon le vecteur d'entrée. Le shellcode utilisé au labo fait **25 octets, sans octet nul**, et sa vérification autonome donne :

SORTIE RÉELLE — LE SHELLCODE LANCE BIEN /BIN/SH

```
shellcode (25 octets) sur la pile @ 0xffffda80
uid=0(root) gid=0(root) groups=0(root)
SHELL_OK_SHELLCODE
```

7. Fiabiliser l'exploit : la technique `jmp esp`

Pointer l'adresse de retour directement sur le tampon est **fragile** : l'adresse exacte de la pile varie d'une machine et d'une exécution à l'autre *(meltypot, « Jumping to the top of the stack »)*. Technique plus robuste :

1. Trouver, à une adresse fixe (dans le binaire ou une bibliothèque), l'instruction `jmp esp` (octets `ff e4`).
2. Écraser l'adresse de retour par celle de ce `jmp esp`.
3. Disposer le shellcode *juste après* l'adresse de retour.

Au `ret`, `eip` = `jmp esp` ; or à cet instant `esp` pointe précisément sur les octets qui suivent l'adresse de retour — c'est-à-dire sur notre shellcode. `jmp esp` y saute. Plus besoin de connaître l'adresse de la pile. La trace réelle du labo 2.6 confirme le mécanisme :

TRACE GDB RÉELLE — JMP ESP → SHELLCODE → EXECVE

```
AT JMP: esp=0xffffda70
mem@esp= 31c050682f2f7368           <- esp pointe sur le shellcode
page esp: ...-... rwxp ... [stack]  <- pile executable (-z execstack)
after jmp pc=0xffffda70             <- jmp esp a saute dans le shellcode
execve path=/bin//sh                <- execve("/bin//sh") atteint
```

8. Les protections mémoire (et pourquoi on les désactive au labo)

Les systèmes modernes activent par défaut plusieurs défenses *(meltypot, « Buffer Overflow Defenses »)*. Elles sont étudiées en détail en **Unité 4** ; ici, un survol pour comprendre pourquoi les binaires du labo sont compilés en les *désactivant*.

Protection	Principe	Drapeau de désactivation (labo)
Stack canary	Valeur aléatoire avant l'adresse de retour ; vérifiée à la sortie de fonction.	`-fno-stack-protector`
NX / DEP	Pile non exécutable : le shellcode sur la pile ne s'exécute pas.	`-z execstack` (rend la pile exécutable)
ASLR	Adresses (pile, bibliothèques) randomisées à chaque exécution.	`echo 0 > /proc/sys/kernel/randomize_va_space`
PIE	Exécutable à adresse de base randomisée.	`-no-pie -fno-pie`

Cadre pédagogique. On désactive ces protections uniquement pour *observer* les mécanismes en laboratoire isolé. En Unité 4, on les réactive et l'on étudie comment elles rendent l'exploitation difficile — et quelles techniques (ret-to-libc, ROP) permettent parfois de les contourner.

9. Remédiations et développement sécurisé

- **Bornes systématiques** : `fgets`, `snprintf`, `strncpy` en contrôlant la taille ; bannir `gets` / `strcpy`.
- **Garder les protections actives** en production (canary, NX, ASLR, PIE, RELRO).
- **Outils** : compiler avec les *sanitizers* (AddressSanitizer — Unité 5) et passer un SAST (Unité 9) pour détecter ces bugs tôt.
- **Langages sûrs en mémoire** (Rust — Unité 6) : la classe entière de bugs disparaît par construction.

10. Points clés

- La pile mélange données (tampons) et contrôle (adresse de retour) : c'est la faiblesse exploitée.
- Un débordement écrase, dans l'ordre, les octets au-delà du tampon : variables voisines → EBP sauvé → adresse de retour.
- Contrôler l'adresse de retour = contrôler `eip` = exécuter du code arbitraire (shellcode).
- Le NOP sled et `jmp esp` fiabilisent le saut vers le shellcode.
- Canary, NX, ASLR, PIE compliquent l'attaque ; on les désactive au labo pour comprendre, on les garde en production.

Références : Aleph One, « Smashing the Stack for Fun and Profit », Phrack 49 (1996) ; support de référence `meltypot_softwaresecurity` (études de cas stack0–stack3, « Getting a shell », « Jumping to the top of the stack », deck « Software Security »).